

PROGRAMAÇÃO MODULAR

SOLID:

OPEN/CLOSED PRINCIPLE

LISKOV SUBSTITUTION PRINCIPLE

PROF. JOÃO CARAM

SOLID PRINCIPLES¹

2

S Single responsibility

O *Open/Closed*

L *Liskov substitution*

I *Interface segregation*

D *Dependency inversion*

¹ - FEATHERS, Michael, a partir de:

MARTIN, Robert C. **Agile software development: principles, patterns, and practices**. Upper Saddle River, N.J.: Prentice Hall, c2003

MARTIN, Robert C; MARTIN, Micah. **Princípios, padrões e práticas ágeis em C#**. Porto Alegre: Bookman, 2011.

SINGLE RESPONSIBILITY - SRP

3

“Uma classe deve ter apenas um motivo para mudar.”

- Uma classe deve ter uma única responsabilidade.
 - *Responsabilidade*: motivo para mudança.
- Conceitos relacionados:
 - Coesão, encapsulamento, abstração.

4

OPEN/CLOSED PRINCIPLE - OCP

ABERTO/FECHADO

OPEN/CLOSED PRINCIPLE - OCP

5

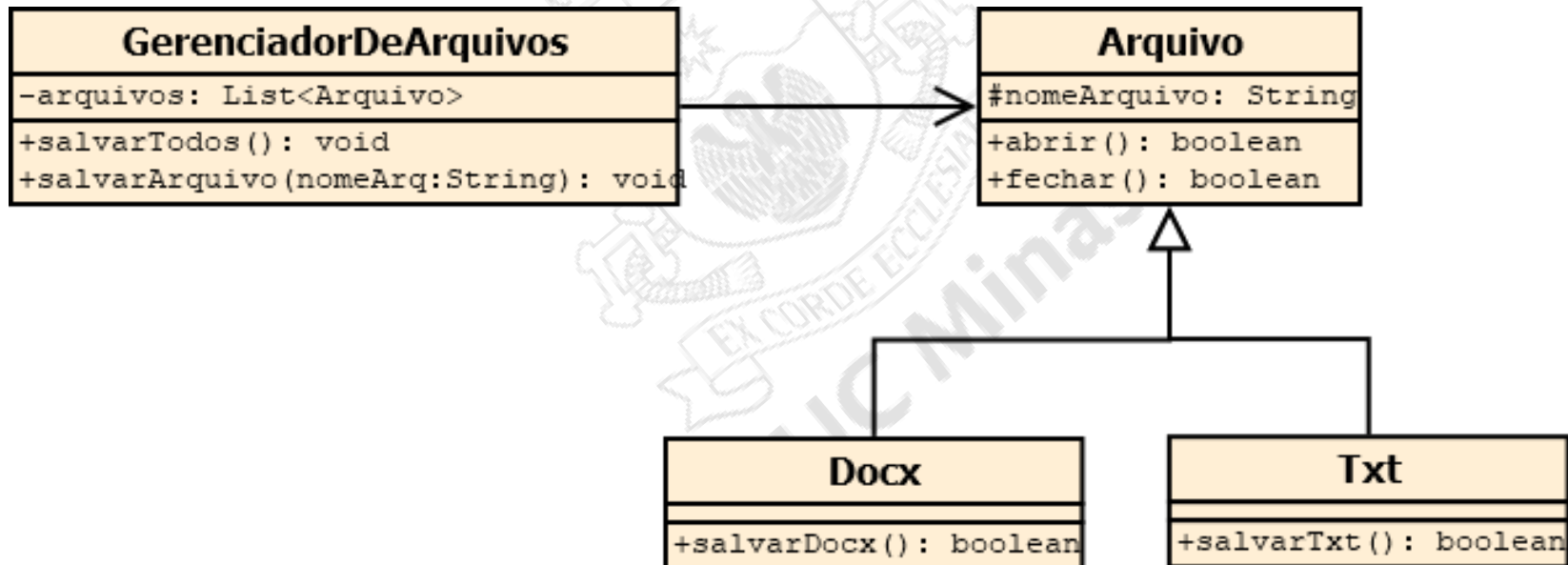
“Um software que funciona deve ser aberto para extensão, mas fechado para modificação.”

- Mudanças apenas para correção de código.
- Extensão: classes abstratas e interfaces.
 - Conceitos relacionados: herança e polimorfismo.

OPEN/CLOSED PRINCIPLE - OCP

Ex: gerenciador de arquivos de vários formatos

Viola o OCP: necessidade de mudança no gerenciador para cada formato



OPEN/CLOSED PRINCIPLE - OCP

```
public class GerenciadorDeArquivos{
```

```
...
```

```
public void salvarTodos(){
```

```
    for(Arquivo arq : arquivos){
```

```
        switch(arq.getClass()){
```

```
            case Docx.class:
```

```
                arq.salvarDocx();
```

```
                break;
```

```
            case Txt.class:
```

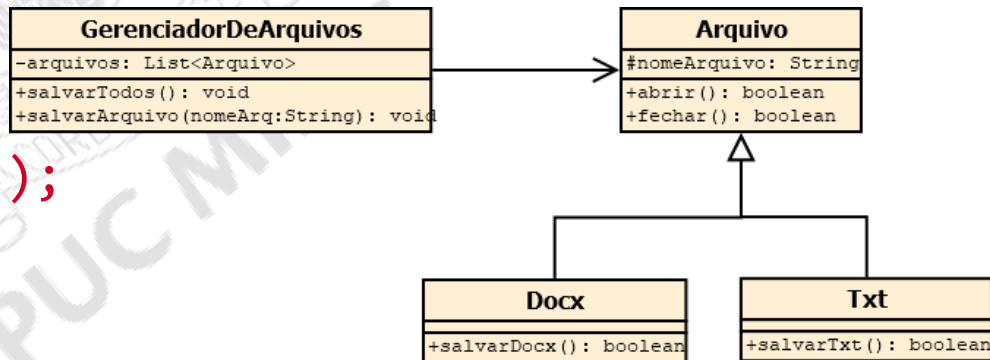
```
                arq.salvarTxt();
```

```
                break;
```

```
        }
```

```
    }
```

```
}
```



OPEN/CLOSED PRINCIPLE - OCP

```
public class GerenciadorDeArquivos{
```

```
...
```

```
public void salvarTodos(){
```

```
for(Arquivo arq : arquivos){
```

```
switch(arq.getClass()){
```

```
case Word.class:
```

```
arq.salvarDocx();
```

```
break;
```

```
case Txt.class:
```

```
arq.salvarTxt();
```

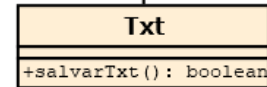
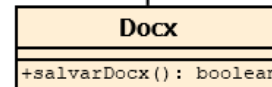
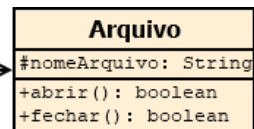
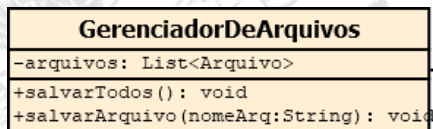
```
break;
```

```
}
```

```
}
```

E se surgirem novos tipos
de arquivo?

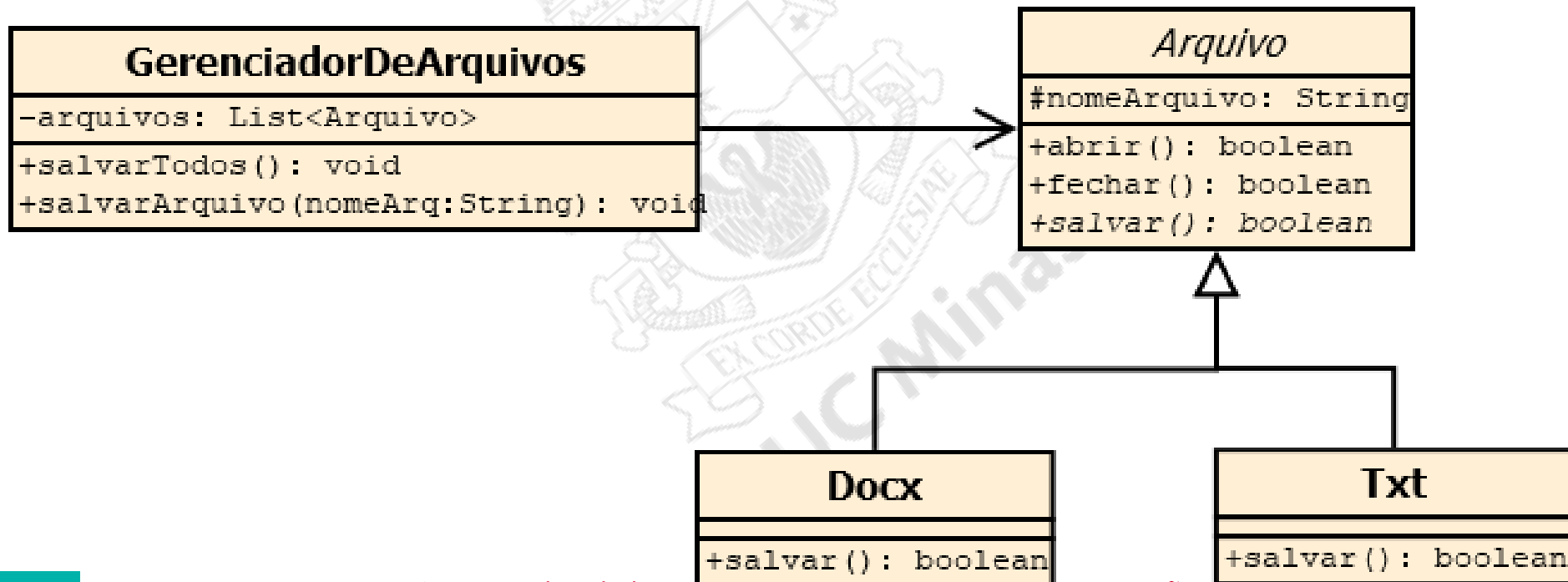
E se cada um criar classes
para arquivos à sua
maneira?



OPEN/CLOSED PRINCIPLE - OCP

Ex: gerenciador de arquivos de vários formatos

Obedece o OCP: abstração e gerenciador estável



OPEN/CLOSED PRINCIPLE - OCP

```
public class GerenciadorDeArquivos{
```

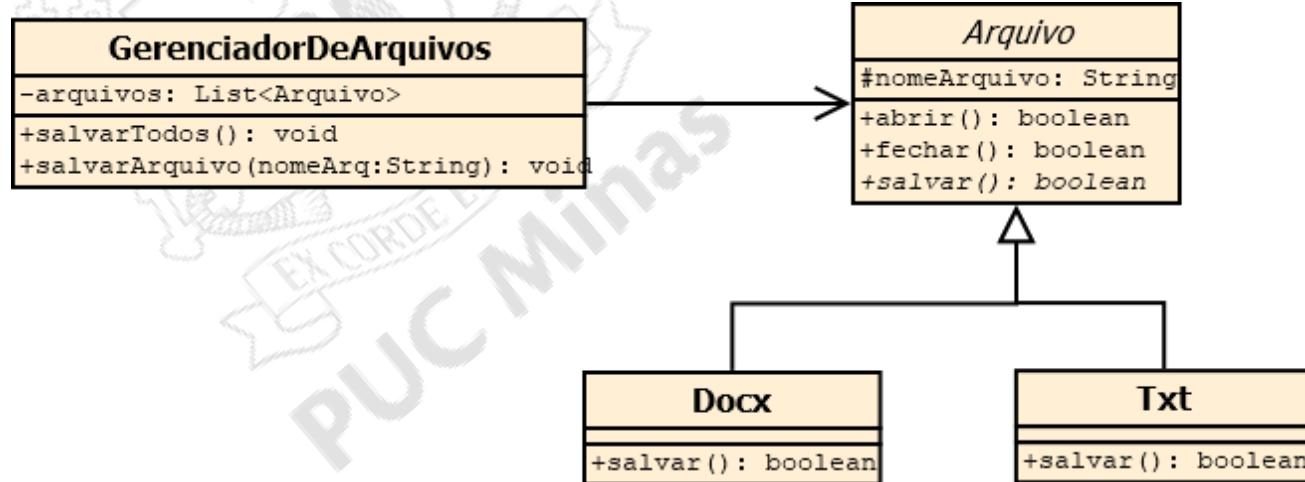
```
...
```

```
public void salvarTodos(){  
    for(Arquivo arq : arquivos){  
        arq.salvar();  
    }  
}
```

```
}
```

```
}
```

```
}
```



OPEN/CLOSED PRINCIPLE - OCP

Violação comum do *OCP*:

Reescrevendo o pedido com novas regras

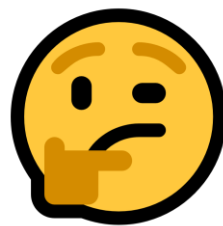
```
class Pedido{  
    private int tipo;  
    private double distancia;  
    public Pedido(int tipo, double dist){  
        init(tipo, dist);  
    }  
    ...  
}
```

OPEN/CLOSED PRINCIPLE - OCP

Violação comum do OCP:

Reescrevendo o pedido com novas regras

```
class Pedido{  
    private double taxa(){  
        if(tipo==1)  
            return valorItens()*0.1;  
        else{  
            if(distancia>10) return 8.0;  
            else if(distancia>5) return 5.0;  
            else return 0.0;  
        }  
    }  
}
```



OCP E NOVOS PEDIDOS

13

- Haverá agora um pedido “promocional”. Neste caso, o cliente pode fazer um pedido para entrega pagando metade do valor da taxa.

OPEN/CLOSED PRINCIPLE - OCP

Violação comum do OCP:

Reescrevendo o pedido com novas regras

```
private double taxa(){  
    double taxa = 0d;  
    switch(tipo){  
        case 1 -> taxa = valorItens()*TAXA_SERVIÇO;  
        case 2 -> taxa = valorEntrega(distancia);  
        case 3 -> taxa = valorEntrega(distancia) * DESCONTO;  
    }  
    return taxa;  
}
```

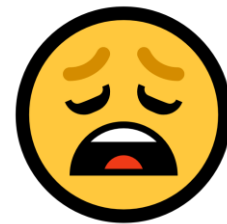


OPEN/CLOSED PRINCIPLE - OCP

Violação comum do OCP:

Reescrevendo o pedido com novas regras

```
private double taxa(){  
    double taxa = 0d;  
    switch(tipo){  
        case 1 -> taxa = valorItens()*TAXA_SERVIÇO;  
        case 2 -> taxa = valorEntrega(distancia);  
        case 3 -> taxa = valorEntrega(distancia) * DESCONTO;  
    }  
    re  
}
```

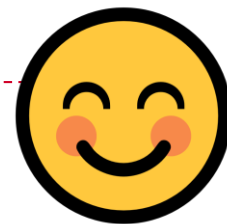


E se surgirem novos tipos
de pedido?

E a responsabilidade única?

OPEN/CLOSED PRINCIPLE - OCP

Em concordância com OCP: pedido não muda

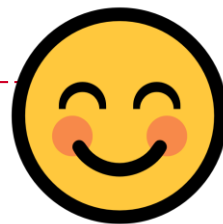


```
abstract class Pedido {  
    ...  
    protected abstract double taxa();  
}
```


OPEN/CLOSED PRINCIPLE - OCP

Em concordância com OCP: pedido não muda

Pedido novoPedido = new PedidoPromocional();

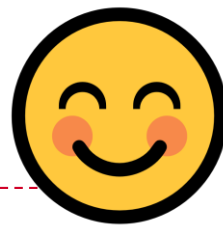


```
class PedidoPromocional extends Pedido {  
    @Override  
    protected double taxa(){  
        return taxaEntrega() * DESCONTO;  
    }  
}
```

OPEN/CLOSED PRINCIPLE - OCP

Em concordância com OCP: pedido não muda

```
Pedido novoPedido = new PedidoPromocional();  
double taxa = novoPedido.precoFinal();
```



```
Pedido novoPedido = new PedidoLocal();  
double taxa = novoPedido.precoFinal();
```

Cada um calcula sua taxa à sua maneira e não precisamos nos preocupar.

19

LISKOV SUBSTITUTION PRINCIPLE - LSP

SUBSTITUIÇÃO DE LISKOV

BARBARA LISKOV

20



Barbara Liskov

(<http://www.pmg.csail.mit.edu/~liskov/>)

Professora do MIT desde 1972.
Departamento de Engenharia Elétrica e
Ciência da Computação desde 2008.

- Medalha John von Neumann da IEEE em 2004;
- Prêmio Turing em 2008;
- Hall da fama dos inventores dos EUA em 2012;
- Prêmio Pioneira da Computação em 2018;
- Medalha Benjamin Franklin em 2023.

LSKOV¹ SUBSTITUTION - LSP

21

“Objetos que fazem parte de um sistema de software podem ser substituídos por instâncias de seus subtipos sem prejuízo para a correção do sistema.”

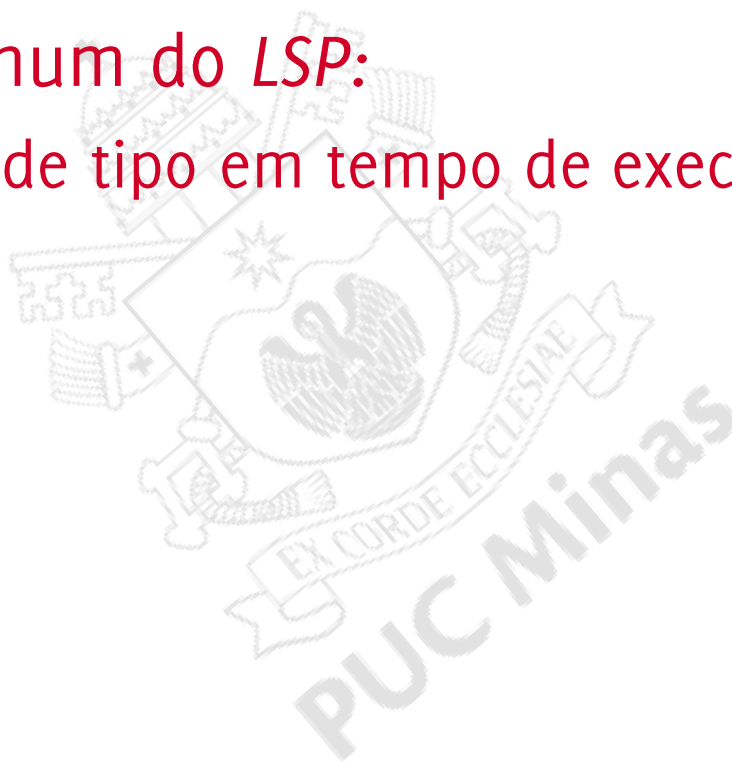
- Conceitos relacionados:
 - herança, polimorfismo e abstração.

¹ – LISKOV, Barbara: *Data abstraction and hierarchy*, 1987

LSKOV SUBSTITUTION - LSP

22

- ▣ Violação comum do *LSP*:
 - ▣ verificação de tipo em tempo de execução.



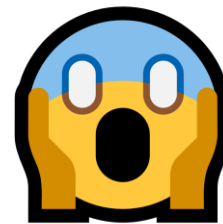
LSKOV SUBSTITUTION - LSP

23

- ▣ Violação comum do *LSP*:
 - ▣ verificação de tipo em tempo de execução.
- ▣ Crime grave!!!!

LSKOV SUBSTITUTION - LSP

Violação comum do *LSP*:
verificação de tipo em tempo de execução



No App:

```
if(comida.getClass().equals(Pizza.class))  
    comida.setPrecoBase(29d);  
else  
    if(comida.getClass().equals(Sanduiche.class))  
        comida.setPrecoBase(13d);
```

Violando OCP no App!!!

LSKOV SUBSTITUTION - LSP

Em concordância com LSP

```
Comida comida = new Pizza();
```

```
class Pizza() extends Comida{  
    private static final double PRECO_BASE = 29d;  
    public Pizza(){  
        super("Pizza");  
        this.setPrecoBase(PRECO_BASE);  
    }  
}
```



LSKOV SUBSTITUTION - LSP

Em concordância com LSP

```
Comida comida = new Pizza();
```

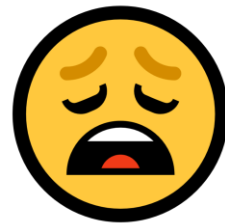
```
Comida outraComida = new Sanduiche();
```

```
class Sanduiche() extends Comida{  
    private static final double PRECO_BASE = 15d;  
    public Sanduiche(){  
        super("Sanduiche");  
        this.setPrecoBase(PRECO_BASE);  
    }  
}
```



LSKOV SUBSTITUTION - LSP

Violação comum do *LSP*:
verificação de tipo em tempo de execução



No App:

```
if(pedido.getTipo()==1)
    pedido.setTaxa(pedido.valorItens()*0.10);
else
    if(pedido.getTipo()==2)
        pedido.calcularTaxaEntrega(distancia);
```

Violando SRP e OCP no App!!!

LSKOV SUBSTITUTION - LSP

Em concordância com LSP

```
Pedido pedido = new PedidoEntrega(dist);
```

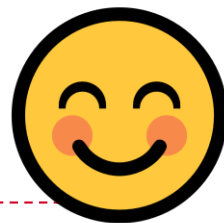


```
class PedidoEntrega() extends Pedido{  
    public PedidoEntrega(double distancia){  
        super();  
        setDistancia(distancia);  
    }  
}
```

LSKOV SUBSTITUTION - LSP

Em concordância com LSP

```
Pedido pedido = new PedidoEntrega(dist);  
pedido = new PedidoLocal();
```

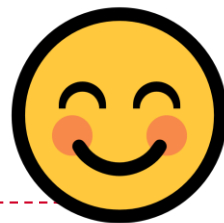


```
class PedidoLocal() extends Pedido{  
    public PedidoLocal(){  
        super();  
    }  
}
```

LSKOV SUBSTITUTION - LSP

Em concordância com LSP

```
Pedido pedido = new PedidoEntrega(dist);  
pedido = new PedidoLocal();  
double aPagar = pedido.precoFinal();
```



```
abstract class Pedido {  
    public abstract double precoFinal();  
}
```

LSKOV SUBSTITUTION - LSP

Em concordância com LSP

```
Pedido pedido = new PedidoEntrega(dist);  
pedido = new PedidoLocal();  
double aPagar = pedido.precoFinal();
```



```
class PedidoEntrega extends Pedido {  
    @Override  
    public double precoFinal(){  
        return valorItens() + valorTaxa();  
    }  
}
```

LSKOV SUBSTITUTION - LSP

32

- Violação sutil: uso da herança desconsiderando o comportamento.
 - Ex: quadrado herdando de retângulo.

LSKOV SUBSTITUTION - LSP

33

- Violação sutil: uso da herança desconsiderando o comportamento.
 - Ex: quadrado herdando de retângulo.
- O crime continua grave, porém requer um pouco mais de experiência para ser percebido.

LSKOV SUBSTITUTION - LSP

Violação sutil do *LSP*:
desconsiderando regras de comportamento

```
Rectangle x = new Square(5);  
x.setHeight(8);
```

LSKOV SUBSTITUTION - LSP

Violação sutil do *LSP*:
desconsiderando regras de comportamento

```
Rectangle x = new Square(5);  
x.setHeight(8);
```

LSKOV SUBSTITUTION - LSP

Violação sutil do *LSP*:

desconsiderando regras de comportamento

Violação do LSP: regra não respeitada

```
Rectangle x = new Square(5);
```

```
x.setHeight(8);
```

Sobrescrever o “setHeight”
do quadrado?

LSKOV SUBSTITUTION - LSP

Violação sutil do *LSP*:

desconsiderando regras de comportamento

Violação do LSP: regra não respeitada

```
Rectangle x = new Square(5);
```

```
x.setHeight(8);
```

...

```
public double dobrarAltura(Rectangle x){
```

```
    x.setHeight(x.getHeight()*2);
```

```
    return x.area();
```

```
}
```

Sobrescrever o “setHeight”
do quadrado?

“x” é quadrado ou
retângulo? Qual regra vale?

LSKOV SUBSTITUTION - LSP

38

- Ex: quadrado herdando de retângulo.
- Em resumo: herança deve considerar “é-um” para tudo o comportamento do objeto.

OBRIGADO.

DÚVIDAS?

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE